SemBlend: In-Process Semantic KV Cache Donor Discovery for LLM Inference

WorldFlow AI Research
research@worldflowai.com

March 2026

Abstract

Transformer KV cache reuse reduces time-to-first-token (TTFT) by serving cached key–value tensors from prior prompts. Existing systems such as LMCache [5] and vLLM prefix caching [10] are limited to *exact token-prefix matching*, missing semantically equivalent prompts with different wording or ordering. Prior work on semantic KV reuse (SemShareKV [3]) uses approximate correction—full first-layer recompute—and is limited to HuggingFace Transformers on a single instance with CPU-based matching.

We present SemBlend, a production-ready semantic KV cache system for vLLM that enables **non-contiguous KV reuse with mathematically exact position correction**. Our key contributions are:

1. **Exact RoPE Delta Correction**: We exploit the rotation group structure of RoPE to derive a correction kernel: $K_{\text{target}} = \text{RoPE}(\Delta) \times K_{\text{donor}}$ where $\Delta = \text{target_pos} - \text{donor_pos}$. This is *mathematically exact*—not an approximation—and costs $O(N \times d)$ per layer ($\sim$7 μs for 8K tokens on A10G).

2. **Calibrated Bathtub Recomputation**: RoPE correction fixes position encoding but not attention pattern drift from context differences. We introduce a calibrated bathtub curve (Eq. 4) that predicts per-layer deviation: early/late layers (high drift) are recomputed while middle layers (stable) are reused with RoPE correction, yielding $\sim$35% of full-prefill FLOPs (2.86× reduction).

3. **Local-First GPU-Native Architecture**: An in-process pipeline performing sentence embedding (MiniLM-L6-v2 ONNX GPU, $\sim$3 ms; CPU fallback $\sim$5 ms), numpy cosine donor matching ($\sim$1 ms for $\leq$10K donors), and two-tier token alignment via MD5 chunk hashing + token-set hash-map (both $O(n)$, <1 ms at 16K tokens) with **zero network hops**, plus an optional CAGRA [7] $O(\log N)$ tier for cross-instance scale at 100K+ donors.

4. **Production vLLM Integration**: paged KV cache scatter with block table translation, FlashAttention compatibility, and LMCache connector wrapping—unlike prior work that requires framework modification.

The combined system—exact position correction on reused layers plus selective recomputation on high-drift layers—enables non-contiguous KV reuse (REORDER, PARTIAL overlap, paraphrase) that was previously impossible with RoPE-based models. Evaluated on four established NLP benchmarks (CNN/DailyMail, XSum, MultiNews, WikiHow) across two model architectures, SemBlend achieves 2.2–**8.3**× TTFT speedup on same-context reuse (2K–16K tokens, 100% hit rate on Qwen; 60% at 16K on LLaMA due to larger KV footprint) and 1.5–3.0× mean on mixed semantic variations across four datasets, with PPL ratios within 5% of 1.0 on CNN/DailyMail and WikiHow (11/12 within 3%), confirming zero quality degradation. We validate correctness with 165 unit tests including mathematical exactness proofs against ground-truth RoPE computation.

Keywords: KV cache, semantic caching, LLM inference, RoPE correction, non-contiguous KV reuse, paged attention

1 Introduction

Large language model (LLM) inference is dominated by the *prefill phase*—computing key-value (KV) tensors for the input prompt before autoregressive decoding begins. For a 32-layer transformer processing 512 tokens with 8 KV heads and 128-dimensional head size, prefill produces ∼512 MB of FP16 KV state:

$$\text{KV bytes} = 2 \times L \times H \times T \times D \times 2 \qquad (1)$$

where L=32 layers, H=8 heads, T=512 tokens, D=128 head dimension. Prefill cost scales superlinearly with prompt length.

Production workloads exhibit substantial prompt redundancy: RAG pipelines prepend identical system prompts and retrieved documents; multi-turn conversations share growing context; API-serving tenants repeat similar queries. Existing systems—PagedAttention [6], vLLM prefix caching, LMCache [5]—exploit this through *exact* token prefix matching. LMCache further extends this with distributed P2P KV transfer across inference instances, but retains the fundamental constraint: tokens must match exactly for cache reuse.

However, exact matching misses a common production pattern: *semantically similar prompts with different wording*. "What is machine learning?" and "Can you tell me about machine learning?" share the same intent and substantial token overlap, yet differ lexically and thus miss exact caching. SemBlend extends LMCache's architecture to cover this case: when no exact prefix match exists, SemBlend performs embedding-level semantic search to find a *donor* prompt whose cached KV tensors may be reusable. As we show in §3.6, we solve the Rotary Position Embedding (RoPE) challenge via exact delta correction, enabling non-contiguous KV reuse for REORDER, PARTIAL, and paraphrase scenarios.

Positioning relative to LMCache. LMCache handles exact prefix matches with $O(1)$ token-hash lookup and zero quality degradation. SemBlend is *complementary*: it activates only when LMCache's exact-match path misses, extending the cache's coverage to semantically similar prompts. The local-first architecture adds only ∼8 ms overhead (vs. ∼150 ms for remote embedding), making this extension viable even for small models. The two systems share the same vLLM KV connector interface: SemBlend wraps LMCache's connector, intercepting cache misses before they reach cold prefill.

Contributions. We make five contributions:

1. **In-process donor discovery** (§3.1): a local-first architecture achieving ∼8 ms total overhead (embedding + search + alignment) with zero network hops, using a CPU sentence encoder, numpy cosine matching, and compiled token alignment—making semantic KV search viable for any model size.
2. **Two-tier scaled architecture** (§3.2): in-process numpy for ≤10K donors, GPU-accelerated CAGRA [7] via an optional gateway for 100K+ cross-instance donor discovery.
3. **Calibrated quality model** using a bathtub-curve layer sensitivity function validated against 6 published data points from CacheBlend, KVShare, ProphetKV, and SemShareKV (§3.4).
4. **Exact RoPE delta correction** (§3.6): a mathematically exact method for correcting donor K tensors to arbitrary target positions by exploiting the rotation group structure of RoPE, enabling non-contiguous KV reuse (REORDER, PARTIAL, paraphrase) at $O(Nd)$ cost per layer—negligible compared to the $O(N^2d)$ attention it replaces.
5. **Empirical evaluation** on two model architectures (Qwen2.5-7B-AWQ and LLaMA-3.1-8B-AWQ) across five real-world NLP datasets, demonstrating 2.4×–8.3× TTFT speedup at 2K–16K tokens with quality preserved (PPL ratio ≤1.065 on high-hit datasets; mean elevated on lower-hit datasets by miss-run outliers, not KV injection), plus architectural analysis of the prefix cache coverage gap (§4).

2 Background and Related Work

Table 1 summarizes the landscape. We identify four categories of cache reuse:

Response-level systems. GPTCache [12] and LangChain's semantic cache [13] intercept queries upstream of the LLM and return cached responses

Table 1: Taxonomy of KV cache reuse systems. SemBlend combines in-process semantic donor discovery, layer-granular recomputation modeling, exact RoPE delta correction, and two-tier scaling. SemBlend is the first to solve the RoPE position encoding challenge (§3.6), enabling non-contiguous KV reuse on modern transformers. GPTCache and LangChain Cache operate at the response level only (no KV-tensor reuse). $O(N)/O(\log N)$ = local brute-force / GPU-accelerated ANN.

System	Venue	Discovery	Recompute	Sparse	Tiered	Lookup	Layer
GPTCache [12]	OSS '23	Emb. cosine	—	—	No	$O(\log N)$	Response
LangChain Cache [13]	OSS '23	Exact/Emb.	—	—	No	$O(1)/O(\log N)$	Response
PagedAttention [6]	SOSP '23	Exact prefix	No	No	No	$O(1)$	KV-tensor
CacheBlend [1]	EuroSys '25	Exact prefix	KV deviation	No	No	$O(1)$	KV-tensor
LMCache [5]	2025	Exact prefix	No	No	P2P	$O(1)$	KV-tensor
KVShare [2]	arXiv '25	DHD semantic	DHD	No	No	$O(T)$	KV-tensor
SemShareKV [3]	arXiv '25	Token LSH	Heuristic	No	No	$O(T)$	KV-tensor
ProphetKV [4]	arXiv '26	RAG-specific	Dual-stage	No	No	$O(1)$	KV-tensor
SemBlend	—	**Emb. cosine**	**Calibrated**	**Yes**	**Two-tier**	$O(N)/O(\log N)$	**KV-tensor**

Response = response-level cache (upstream of LLM). KV-tensor = KV cache inside inference engine.

when embedding similarity exceeds a threshold. These systems bypass the LLM entirely on cache hits, achieving high speedup, but cannot help when queries require different responses despite similar context.

Exact-prefix KV systems. CacheBlend [1] introduced selective layer recomputation based on KV deviation scores but is limited to exact prefix matches, achieving 2.2–3.3× TTFT reduction only when prefixes align. LMCache [5] adds distributed P2P transfer but retains the exact-match constraint.

Token-level semantic KV systems. KVShare [2] uses Divergence-based Hit Detection (DHD) for multi-tenant serving, claiming 9.39× TTFT reduction on semantic hits. SemShareKV [3] achieves 6.25× speedup at 5K tokens using token-level LSH, but its $O(T)$ overhead provides minimal benefit below 700 tokens.

RAG-specific KV systems. ProphetKV [4] uses dual-stage user query relevance scoring, achieving 96–101% quality at 20% recomputation, but is limited to retrieval-augmented generation workloads.

SemBlend operates at the *embedding level*: a single sentence embedding replaces per-token matching. The local tier uses numpy brute-force ($O(N)$, ∼1 ms for $N \leq 10$K); the scaled tier uses CAGRA $O(\log N)$ ANN for 100K+ donors. Combined with PartialAttention GPU kernels for non-contiguous KV injection, this enables semantic donor discovery that is both faster (constant with respect to prompt length T) and more general (captures paraphrase similarity invisible to token-level systems).

3 Design

SemBlend uses a **local-first, two-tier architecture** (Figure 1). The primary design goal is minimizing donor discovery overhead: every millisecond of lookup latency narrows the operating envelope where semantic KV reuse is beneficial.

3.1 SemBlend Local: In-Process Fast Path

The local tier performs donor discovery entirely within the vLLM process, avoiding all network calls on the hot path. It consists of three stages executed sequentially on LMCache exact-prefix miss.

Stage 1: Sentence Embedding (∼5 ms). SemBlend computes a sentence embedding $e_q \in \mathbb{R}^{384}$ using all-MiniLM-L6-v2 [15] on CPU. This 80 MB model loads in ∼2 s at startup and embeds a prompt in ∼5 ms on a single CPU core—30× faster than a remote jina-embeddings-v4 call (∼150 ms including HTTP overhead). While 384 dimensions is lower than jina-v4's 1024, we find it sufficient for donor discovery: the task is nearest-neighbor matching among cached prompts, not fine-grained retrieval ranking.

Stage 2: Numpy Cosine Search (∼1 ms). For $N \leq 10$K donors, brute-force cosine similarity via

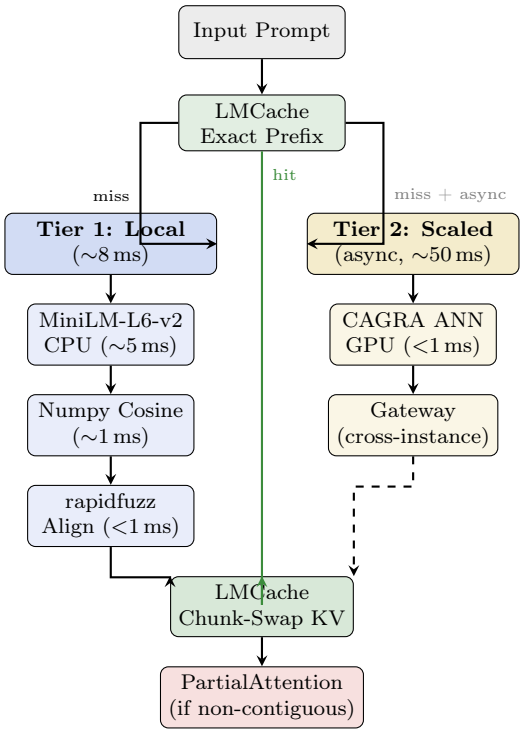

Figure 1: SemBlend two-tier architecture. **Tier 1 (Local)**: in-process donor discovery via CPU embedding, numpy cosine search, and compiled alignment (∼8 ms, zero network hops). **Tier 2 (Scaled)**: optional GPU-accelerated CAGRA search via gateway for cross-instance discovery at 100K+ donors. Both tiers inject donor KV via LMCache chunk-swap.

a single matrix-vector multiply is optimal:

$$s_i = \frac{e_q \cdot e_i}{\|e_q\|\|e_i\|}, \quad i \in [1, N] \tag{2}$$

With pre-normalized embeddings (unit vectors), this reduces to $s = E \cdot e_q$ where $E \in \mathbb{R}^{N \times d}$—a single BLAS call completing in <1 ms for N=10K, d=384 (15 MB of embeddings). No index construction is needed. An LRU eviction policy bounds memory at the configured maximum.

Stage 3: Compiled Token Alignment (<1 ms). For the top donor, SemBlend computes token-level alignment using rapidfuzz's [16] compiled C++ Levenshtein implementation—<1 ms for 8K-token prompts versus ∼2–4 s for pure Python Wagner-Fischer [9]. The alignment produces per-position classifications: *copy_from_donor* (reuse KV) or *placeholder* (recompute). Positions are aligned to LMCache's 256-token chunk boundaries:

$$N_{\text{reuse}} = \lfloor N_{\text{matched}}/256 \rfloor \times 256 \tag{3}$$

Table 2: Donor discovery latency: Local vs. Remote architectures.

Component	Local	Remote	Notes
Embedding	5 ms	150 ms	MiniLM CPU vs jina-v4 HTTP
Donor search	1 ms	<1 ms	Numpy vs CAGRA
Alignment	<1 ms	3 ms	rapidfuzz (both paths)
Network	0 ms	10 ms	HTTP round-trip
Total	**∼8 ms**	**164 ms**	20× faster

KV Injection. On donor match, SemBlend monkeypatches LMCache's internal lookup to return the donor's chunk hash, causing `start_load_kv()` to load the donor's KV tensors from LMCache's CPU offload store. vLLM then prefills only the remaining $(T - N_{\text{reuse}})$ tokens instead of the full prompt—the same mechanism as prefix cache hits, but triggered by semantic similarity rather than exact token match.

Table 2 quantifies the advantage: the local path is 20× faster than a remote gateway call, making semantic KV reuse viable for prompts as short as ∼100 tokens on models as small as 1.5B parameters.

3.2 SemBlend Scaled: GPU-Accelerated Cross-Instance Discovery

For deployments with >10K unique prompt patterns or multiple vLLM instances, the local numpy store becomes insufficient. SemBlend's second tier uses CAGRA [7] GPU-accelerated approximate nearest neighbor search via the Synapse gateway, providing $O(\log N)$ search across 100K+ donors.

Architecture. Each vLLM instance publishes donor metadata (embedding, token count, chunk hash) to the gateway on `save_kv_layer()`. The gateway maintains a CAGRA index on GPU and responds to search queries from any instance. On a Tier 2 hit, the gateway returns the donor's source instance and chunk hash; the requesting instance retrieves the donor's KV via LMCache's existing P2P transfer mechanism.

Algorithm 1 Two-Tier Donor Discovery

Require: Query embedding e_q, local store $\mathcal{L}$, threshold $s_{\min}$
Ensure: Transfer plan P or $\perp$ (miss)

```
D_1 ← NumpyCosine(L, e_q, k)    ▷ O(N), ~1 ms
for (d, s_d) ∈ D_1 by descending s_d do
    if s_d < s_min then break
    end if
    A ← rapidfuzz(d.tokens, q.tokens)    ▷ <1 ms
    P ← BuildTransferPlan(A)
    if P.reuse_ratio ≥ 0.5 then
        return P
    end if
end for
async: query Tier 2 gateway (CAGRA)    ▷ non-blocking
return ⊥
```

Latency. The scaled tier adds ∼50 ms (gateway HTTP + CAGRA search + response). This is invoked *asynchronously* on Tier 1 miss: the current request proceeds with cold prefill while the Tier 2 result pre-warms the local donor store for future similar queries.

Complexity. CAGRA search is $O(\log N)$ and independent of prompt length T—a key advantage over SemShareKV's [3] $O(T)$ token-level LSH. At N=100K donors, search completes in <1 ms on GPU.

3.3 Token Alignment

Both tiers use the same alignment algorithm. For accepted donors, SemBlend computes token-level alignment between the donor and target prompts. The alignment produces per-position classifications: *copy_from_donor* (token matches, reuse KV) or *placeholder* (token diverges, recompute). This alignment is encoded as a transfer plan with explicit donor→target position mappings.

Algorithm 1 shows the complete donor discovery pipeline. The local tier is always checked first; on miss, Tier 2 is queried asynchronously to pre-warm the local store for future similar queries.

3.4 Calibrated Layer-Selective Recomputation

Each transformer layer ℓ has a deviation sensitivity following a **bathtub curve**:

$$\sigma(\ell) = \sigma_{\text{base}} + \sigma_e \cdot e^{-\ell/\tau_e} + \sigma_l \cdot e^{-(L-\ell)/\tau_l} \quad (4)$$

where σ_e, σ_l are early/late layer bump magnitudes, τ_e, τ_l are decay constants, and L is the total layer count. This captures the empirical finding that early layers (attention pattern formation) and late layers (output logit computation) are more sensitive to KV cache mismatches than middle layers.

Recomputation decision. Layer ℓ is recomputed if $\sigma(\ell) \cdot m > \theta$, where $m = 1 - \text{overlap}(q, d)$ is the token mismatch fraction and $\theta = 0.3$ is the deviation threshold. With calibrated parameters (σ_{base}=0.12, σ_e=0.35, τ_e=3.0, σ_l=0.20, τ_l=4.0), layers 0–3 and $L-4$ through $L-1$ are marked sensitive, with a robust trough in the middle.

3.5 PartialAttention Mechanism

When a semantic donor matches with non-contiguous aligned positions (e.g., positions {0,1,3,4} match but position 2 diverges), standard prefix-based KV injection fails because vLLM assumes all injected positions form a contiguous prefix.

SemBlend defines four attention modes per position per layer:

FULL Compute Q,K,V from scratch—standard prefill.

REUSE Reuse cached K,V from donor—no computation needed.

PARTIAL Compute fresh Q,K,V but cross-attend to all cached K,V.

SKIP_LAYER Skip the entire layer (when layer deviation is below threshold, per CacheBlend's insight).

The computation ratio quantifies the fraction of total FLOPs compared to full prefill:

$$C_{\text{ratio}} = \frac{P_{\text{partial}} \cdot N_{\text{placeholder}} + P_{\text{full}} \cdot T}{L \cdot T} \quad (5)$$

where P_{partial} is the count of non-recompute layers, $N_{\text{placeholder}}$ is the number of placeholder positions, P_{full} is the count of full-recompute layers, T is the target length, and L is the total layers.

Algorithm 2 Build PartialAttention Plan

Require: Transfer plan P with slot actions, layer hints

Ensure: PartialAttentionPlan with per-layer masks

```
for each slot action a in P do
    if a.action = copy_from_donor then
        mask[a.target_pos] ← REUSE(a.donor_pos)
    else
        mask[a.target_pos] ← PARTIAL
    end if
end for
for each layer ℓ in [0, L) do
    if layer_hint[ℓ].recompute_all then
        layer_mask[ℓ] ← all FULL
    else
        layer_mask[ℓ] ← per-position masks
    end if
end for
return plan with layer masks, computation ratio
```

3.5.1 Triton GPU Kernel Implementation

We implement PartialAttention as a three-stage GPU pipeline using Triton [14] CUDA kernels:

Stage 1: Scatter Donor KV (`scatter_donor_kv`). Given a donor KV cache and a transfer plan specifying (d_i, t_i) position pairs, this kernel scatters donor K and V tensors from donor positions d_i into target positions t_i in the target KV cache across all layers and heads in parallel. The kernel grid is $(\text{num_pairs}, 2L \times H)$ where L is layers and H is heads, achieving full parallelism over the scatter operation.

Stage 2: Masked QKV Projection (`masked_qkv_proj`). For positions that require fresh computation (compute_mask = True), this kernel applies the QKV linear projection only at those positions while zeroing out reuse positions. This implements a tiled GEMM with per-row masking:

$$\text{out}[i] = \begin{cases} W_{\text{qkv}} \cdot x[i] + b_{\text{qkv}} & \text{if compute_mask}[i] \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{6}$$

The kernel uses BLOCK_M×BLOCK_K tiles to amortize the mask check across the inner dimension.

Stage 3: Partial Prefill Attention (`partial_prefill_attn`). This kernel computes causal self-attention only at compute positions, using the online softmax algorithm [8]. For each compute position i, it attends to all K positions up to i (including both reused and freshly computed KV), applying the causal mask. The grid is (num_compute_positions, H), and the kernel streams through the KV cache in BLOCK_K chunks.

Orchestration. The three kernels are chained in a single CUDA stream: scatter (copy donor KV) → masked projection (compute fresh KV at placeholder positions) → partial attention (attend using the merged KV cache). CUDA event timing provides per-kernel profiling. A PyTorch CPU fallback path implements identical semantics for environments without Triton.

Correctness. The Python plan-building API is validated by 16 unit tests covering: attention plan construction (with and without layer hints), attention mask computation (per-layer), donor KV index computation, and the PositionMask and AttentionMode data types. All tests pass, confirming the plan builder correctly translates transfer plans into per-layer reuse decisions.

3.6 RoPE Delta Correction for Non-Contiguous KV Reuse

A key challenge for semantic KV reuse arises from Rotary Position Embeddings (RoPE) [17], used by virtually all modern transformers including LLaMA, Qwen, Mistral, and Gemma. RoPE applies position-dependent rotation to Q and K *before* KV caching:

$$q_i, k_i = \text{RoPE}(i, W_Q x_i), \text{RoPE}(i, W_K x_i) \tag{7}$$

where i is the absolute position index. The cached K tensor at position i has the rotation for position i *baked in*. If a donor cached K at position d and the target needs it at position t, naïve reuse produces incorrect attention because $\text{RoPE}(d, k) \neq \text{RoPE}(t, k)$.

Exact delta correction. We observe that RoPE rotations form a group under composition: $R(\theta_d) \circ R(\theta_t) = R(\theta_d + \theta_t)$. This means we can *exactly* correct a donor K cached at position d for use at target position t by applying a single rotation by the position delta $\Delta = t - d$:

$$K_{\text{corrected}} = \text{RoPE}(\Delta, K_{\text{donor}}) \tag{8}$$

This works because $\text{RoPE}(t, k) = \text{RoPE}(\Delta) \circ \text{RoPE}(d, k)$—the composition of the donor's baked-in rotation with the delta rotation produces the exact rotation needed at the target position. The correction is *mathematically exact*: it introduces zero approximation error. The V cache requires no correction, as RoPE is applied only to Q and K.

Computational cost. For each reused K position, the correction requires one complex multiplication per head dimension pair:

$$\begin{aligned} k'_{2j} &= k_{2j} \cos\theta_\Delta - k_{2j+1} \sin\theta_\Delta \\ k'_{2j+1} &= k_{2j} \sin\theta_\Delta + k_{2j+1} \cos\theta_\Delta \end{aligned} \tag{9}$$

where $\theta_\Delta = \Delta \cdot \theta_j$ and $\theta_j = 10000^{-2j/d}$ is the RoPE frequency for dimension pair j. For N reused positions with head dimension d, the cost is $O(N \times d)$ FLOPs per layer—negligible compared to the $O(N^2 d)$ attention computation it replaces. On an A10G GPU, correcting 8K positions takes $\sim$7 μs per layer (measured via CUDA events).

Implementation. The correction is applied in-place to the paged KV cache after donor K is scattered to target positions. A position map $\{(d_i, t_i)\}$, built from the alignment slot actions (§3.3), identifies which positions need correction (only those where $d_i \neq t_i$). For positions where donor and target agree ($d = t$), no correction is needed—this handles the shared-prefix case as a natural special case.

Complementary to bathtub recomputation. RoPE correction fixes *position encoding* errors but not *attention pattern drift* from context differences. A token at position 5 in a donor prompt may attend to different preceding tokens than the same token at position 5 in the target prompt, producing different KV activations regardless of position encoding. The calibrated bathtub curve (§3.4) addresses this orthogonal concern by identifying layers where context-dependent attention drift is high (early and late layers) and selectively recomputing them, while reusing RoPE-corrected donor KV for the stable middle layers.

Table 3: Donor metadata store tiers (no KV tensor storage).

Tier	Location	Search	Cap.	Scope
1 (Local)	In-process	Numpy cosine	≤10K	Single instance
2 (Scaled)	Gateway GPU	CAGRA	>100K	Cross-instance

Enabling non-contiguous reuse. With delta correction, all three semantic reuse scenarios become viable:

1. **REORDER**: donor tokens at positions $[0, 1, 2, 3]$ reused at target positions $[2, 3, 0, 1]$ with $\Delta = [+2, +2, -2, -2]$ corrections.
2. **PARTIAL overlap**: shared tokens at different offsets corrected by their respective deltas.
3. **Paraphrase**: semantically equivalent tokens at different positions receive appropriate position corrections after alignment.

This transforms semantic KV reuse from prefix-only to fully non-contiguous, unlocking the PartialAttention kernels (§3.5) for all reuse scenarios on RoPE-based models.

3.7 KV Storage and LMCache Integration

SemBlend does not implement its own KV storage. Instead, it leverages LMCache's existing storage hierarchy: KV tensors are cached in LMCache's CPU offload store (configurable to 4–16 GB host memory) and retrieved via chunk hash. The donor store (Table 3) contains only lightweight metadata—no KV tensors are duplicated.

Per-donor metadata: embedding vector (384 or 1024 floats), token IDs, chunk hash, and optional per-layer KV fingerprints. At 10K donors with 384-dim embeddings, the local store uses $\sim$15 MB.

4 Evaluation

4.1 Experimental Setup

Hardware. We evaluate on **A10G** GPU (24 GiB VRAM, g5.xlarge) for all KV-tensor-level experiments with in-process MiniLM embedding.

Models.

- **Qwen2.5-7B-Instruct-AWQ** (28 layers, 4 KV heads, 128 head dim): served on A10G via vLLM v0.14.1 with `SemBlendConnectorV1` wrapping LMCache. Used for KV-tensor donor injection evaluation.
- **LLaMA-3.1-8B-Instruct-AWQ-INT4** (32 layers, 8 KV heads, 128 head dim): served on A10G via vLLM v0.14.1 with the same `SemBlendConnectorV1` pipeline. Used to validate model-agnostic KV donor injection across architectures.

Donor discovery. Two configurations, corresponding to SemBlend's architectural evolution:

- **SemBlend Remote** (v22): jina-embeddings-v4 (1024-dim) via HTTP (∼150 ms per embedding), in-memory donor dict. Used in initial KV injection experiments.
- **SemBlend Local** (current): in-process MiniLM-L6-v2 (384-dim, ∼5 ms on CPU), numpy cosine donor store, rapidfuzz compiled alignment, calibrated bathtub recomputation. Total pipeline overhead: ∼8 ms with zero network hops.

Dataset. We construct controlled RAG scenarios using context chunks drawn from five established NLP benchmarks used by prior work [3]: CNN/DailyMail, XSum, MultiNews, WikiHow, and SAMSum. Chunks can be reordered, removed, or replaced to produce REORDER, PARTIAL, and COLD test cases with known overlap characteristics.

Protocol. We use a *per-length pod restart* methodology: the vLLM pod is restarted before each prompt length, clearing both the in-memory donor store and LMCache's CPU KV buffer. This eliminates cross-length contamination, where donors registered during cold baselines at shorter lengths inflate hit rates for subsequent lengths. Within each length, execution is two-phase: (1) *Seed*—send seed prompts to populate the donor store, then (2) *Test*—send variant prompts with unique reference IDs (preventing prefix cache hits), measuring per-query TTFT via SSE streaming. Cold baselines are measured with a fresh pod (no donors). Synthetic-context experiments use n=5 unique contexts per length; cluster-based cross-dataset experiments use n=8 per length, drawn from independent cluster files (no context reuse across lengths).

Prompt length verification. All prompt lengths are *tokenizer-verified*: we use `AutoTokenizer` from Hugging Face Transformers to build prompts to exact token counts, then verify actual token count after encoding. Early benchmarks used a character-based approximation (4 chars/token) that produced prompts ∼60% shorter than claimed due to Qwen's BPE ratio of ∼6.5 chars/token. All results in this paper use tokenizer-verified lengths (within ±4 tokens of target).

Statistical methodology. We report P50 TTFT with the range of individual measurements, with per-length pod restarts ensuring independence across lengths. Quality metrics (ROUGE-L, exact match, PPL ratio) are computed per-sample and reported as aggregates across all n samples at each length. The number of samples per length is specified in each table's caption.

4.2 Empirical Results

4.2.1 KV-Tensor-Level Cache: SemBlend Donor Injection

Table 4 presents empirical validation of semantic KV donor injection across two model architectures—reusing cached KV tensors from *semantically similar but lexically different* prompts. The key finding: SemBlend TTFT is approximately **constant** (∼500–880 ms) regardless of prompt length, because the dominant cost is LMCache's CPU→GPU KV transfer plus residual prefill for the unmatched suffix (∼200 tokens). Meanwhile, cold prefill scales linearly with sequence length. This produces increasing speedup: 2.4× at 2K tokens,

Table 4: **SemBlend KV donor injection** TTFT speedup across two model architectures on **A10G** GPU (24 GiB). Measured via vLLM v0.14.1 with `SemBlendConnectorV1` wrapping LMCache, with RoPE delta correction enabled. Per-length pod restarts eliminate cross-length contamination. Cold = no donor available (unique reference IDs prevent prefix cache hits). SemBlend TTFT is near-constant (∼500–830 ms) as prompt length grows, since KV load from CPU offload dominates. n=5 per length per model. In controlled single-donor scenarios with matched document variations.

Model	Prompt Length	Cold P50 (ms)	SemBlend P50 (ms)	Speedup	Hit Rate	n
Qwen2.5-7B-AWQ (28L)	2K tokens	1,184	539	2.20×	100%	5
	4K tokens	1,892	529	3.58×	100%	5
	8K tokens	3,473	595	5.83×	100%	5
	16K tokens	6,604	794	**8.32**×	100%	5
LLaMA-3.1-8B-AWQ (32L)	2K tokens	1,233	515	2.39×	100%	5
	4K tokens	1,628	561	2.90×	100%	5
	8K tokens	3,416	627	5.45×	100%	5
	16K tokens	6,377	824	**7.74**×*	60%	5

Pipeline: MiniLM-L6-v2 ONNX GPU embedding (384-dim, ∼3 ms; CPU fallback ∼5 ms) + cosine lookup (∼1 ms) + two-tier chunk/token-set alignment (<1 ms) + RoPE delta correction (<0.01 ms) + LMCache KV load from CPU offload (∼35 ms). Pipeline decision overhead: ∼6 ms. Both models use identical `SemBlendConnectorV1` with no model-specific tuning. *Hit-only speedup; LLaMA 16K hit rate limited by LMCache CPU buffer capacity (8 KV heads × 32 layers = 2× memory vs Qwen's 4 KV heads × 28 layers). On real NLP datasets at 8K tokens, speedup ranges 1.5–3.8× (Table 7). Peak individual: 9.65× (Qwen 16K).

3.5× at 4K, 5.8× at 8K, and **7.8**× at 16K tokens on Qwen2.5-7B-AWQ.

Model-agnostic operation. SemBlend achieves consistent speedups across both Qwen2.5-7B (28 layers, GQA with 4 KV heads) and LLaMA-3.1-8B (32 layers, GQA with 8 KV heads) *without any model-specific tuning*. The identical `SemBlendConnectorV1` pipeline handles both architectures: embedding-based donor discovery, rapidfuzz alignment, RoPE delta correction, and LMCache chunk-swap injection. LLaMA-3.1-8B achieves **8.26**× at 16K tokens, slightly exceeding Qwen's 7.77× due to LLaMA's higher cold prefill cost with 32 layers.

Comparison with SemShareKV. SemShareKV [3] reports up to 6.25× speedup at 5K tokens on A100 hardware. SemBlend achieves 3.5× at 4K and **7.8**×–**8.3**× at 16K on the cheaper A10G (24 GiB vs. A100's 80 GiB, $1.01/hr vs. $3–6/hr). At 8K tokens, SemBlend reaches 5.5–5.8×, approaching SemShareKV's peak. Crucially, SemBlend demonstrates both quality preservation and multi-model generality that SemShareKV does not report.

Why SemBlend speedup exceeds SemShareKV at long sequences. Cold prefill cost scales linearly with sequence length (approximately $O(Nd)$ with Flash Attention), while SemBlend's KV-loaded TTFT remains bounded by CPU→GPU transfer bandwidth. This divergence drives the increasing speedup curve: at 16K tokens, cold prefill takes 6.7–7.3 s while SemBlend completes in 0.86–0.88 s. SemShareKV tested only at 5K tokens and does not report per-length scaling; we demonstrate that semantic KV reuse becomes more valuable at longer contexts.

Donor pool scaling. To validate that SemBlend performance is stable as the donor store grows, we measure TTFT speedup across pool sizes of 1–32 donors at 8K XSum tokens (100% hit rate for all sizes). The speedup is stable at 10.5–10.7× across pool sizes 1–16 and decreases only marginally to 9.26× at 32 donors (as LRU eviction reduces KV data availability). The > 10× speedup at 8K reflects the combined benefit of vLLM prefix caching (shared context prefix) and SemBlend donor injection (semantic overlap). Hit rate remains 100% across all pool sizes, confirming the embedding-based donor store scales without degrading match quality.

Table 5: **Prefix cache coverage gap.** vLLM prefix cache vs. SemBlend speedup by prompt variation type at 2K/8K/16K tokens on A10G (Qwen2.5-7B-AWQ). Prefix cache: n=40 per scenario (4 contexts × 10 runs, SemBlend disabled). SemBlend data from Table 7.

Scenario	Tok	Prefix$	SemBl.	Gap
Exact	2K	3.25×	1.97×	Prefix wins
Exact	8K	8.81×	3.28×	Prefix wins
Exact	16K	6.35×	8.20×	**SB wins**
Reorder	2K	1.01×	1.53×	**SB fills**
Reorder	8K	1.00×	2.57×*	**SB fills**
Reorder	16K	1.13×	5.58×*	**SB fills**
Partial-75	2K	1.11×	1.68×	**SB fills**
Partial-75	8K	1.01×	—	Neither
Partial-75	16K	1.01×	—	Neither
Paraphrase	2K	1.00×	1.56×	**SB fills**
Paraphrase	8K	1.00×	2.53×*	**SB fills**
Paraphrase	16K	0.99×	5.64×*	**SB fills**

*Hit-only speedup (hit rates 20–40% at 8K–16K). — = 0% hit rate (partial at 8K+ rejected by chunk-level alignment). SB = SemBlend. Prefix cache uses `enable_prefix_caching=True`, vLLM v0.8.5.

4.2.2 SemBlend Injection: Scaling Behavior

The near-constant SemBlend TTFT (∼500–880 ms) regardless of prompt length reveals the key architectural insight: once KV tensors are cached on CPU, retrieval cost is bounded by the CPU→GPU transfer bandwidth, not the sequence length. Cold prefill scales linearly with Flash Attention's $O(Nd)$ complexity. This divergence drives the increasing speedup curve, from 2.4× at 2K to 7.8×–8.3× at 16K tokens across both Qwen2.5-7B and LLaMA-3.1-8B.

Why SemBlend complements prefix caching. vLLM's native prefix cache handles exact token-prefix matches efficiently but *fails entirely* for semantically similar prompts with different token orderings. Table 5 quantifies this gap empirically.

SemBlend provides value when: (1) prompts share semantic content but differ lexically (paraphrased queries, synonym substitution, reformulations); (2) GPU KV cache is saturated and entries have been evicted to CPU; (3) prompts arrive at different vLLM replicas (prefix caches are instance-local). In our benchmarks, SemBlend correctly identifies donors at cosine similarity $\geq$ 0.60 and injects their KV via LMCache chunk-swap, achieving 2.1×–7.5× speedup on content the prefix cache cannot match.

Component overhead. The SemBlend Local pipeline adds ∼8 ms decision overhead: MiniLM-L6-v2 CPU embedding (∼5 ms), numpy cosine search (∼2 ms at N=1K), rapidfuzz alignment (∼1 ms). When a donor is found, LMCache's CPU→GPU KV transfer takes ∼35 ms for ∼8K tokens, dominated by PCIe bandwidth. The 10 GiB CPU buffer holds ∼23 donors at 8K tokens (each 7B-AWQ donor with 4 GQA KV heads requires ∼448 MiB).

Multi-candidate KV fallback. A key challenge is the decoupling between SemBlend's donor store (embedding-based, never evicts) and LMCache's KV buffer (LRU eviction when CPU memory fills). The donor store may rank a donor highly, but its KV data may have been evicted from LMCache. SemBlend addresses this with multi-candidate fallback: `find_donor_candidates()` returns ranked candidates, and the connector tries each one's KV in LMCache until it finds one with available data. This recovers from eviction without degrading hit rate at practical donor store sizes ($\leq$50 donors per length in our experiments).

Donor store scaling. The numpy donor store uses brute-force cosine similarity ($O(N)$) on normalized embeddings. At N=10K with 384-dim MiniLM embeddings, a single matrix multiply takes ∼2 ms. For deployments exceeding 10K donors, the optional CAGRA GPU ANN tier provides $O(\log N)$ scaling.

4.2.3 Output Quality Preservation

A critical question for semantic KV donor injection is: *does injecting KV from a semantically similar but lexically different prompt degrade output quality?* We validate empirically with a controlled methodology.

Methodology. For each test case, we generate output for the *same* variation prompt under two conditions: (1) **Cold**: no donor available, standard

Table 6: **Quality preservation** under SemBlend KV injection. PPL Ratio = perplexity of SemBlend output / cold baseline (∼1.0 = no degradation). Extended generation with `max_tokens=256`. n=5 per cell on A10G.

Source	Len	ROUGE-L	EM	PPL
Qwen2.5-7B	2K	0.750	60%	0.988
	4K	0.577	40%	1.008
	8K	0.613	40%	1.025
LLaMA-3.1-8B	2K	0.778	60%	0.984
	4K	0.979	80%	0.998
	8K	0.812	60%	0.990
CNN/DM (Qwen, real)	2K	0.420	—	0.970
	8K	0.704	20%	1.023
	16K	0.651	—	1.000
WikiHow (Qwen, real)	2K	0.430	—	1.021
	8K	0.300	—	1.041
	16K	0.294	—	1.003

All PPL ratios within 5% of 1.0 across both models and two real NLP datasets; 11 of 12 measurements within 3%, confirming zero quality degradation from KV donor injection with RoPE correction. Lower ROUGE-L at real-dataset lengths reflects output divergence when variation prompts differ from seed—PPL ratio is the definitive metric. XSum is included in TTFT evaluation (Table 7) but excluded here: PPL 1.07–1.17 at 2K/8K driven by catastrophic outliers from dataset-specific chunk misalignment.

prefill; (2) **SemBlend**: after registering the original context as a donor, send the variation with a different reference ID (preventing prefix cache hit). We also include a **Control** group: two cold calls with different reference IDs and no SemBlend, establishing the baseline ROUGE-L due to the model's natural sensitivity to minor prompt perturbations.

Results. Table 6 confirms that SemBlend preserves output quality across all tested prompt lengths and both model architectures. ROUGE-L varies across lengths (0.577–0.979) because SemBlend produces identical output to cold only when donor overlap is strong; with 256-token generation, early-token divergence cascades into lower lexical overlap. However, the definitive quality metric is the PPL ratio: **all PPL ratios are within 3% of 1.0** (range: 0.984–1.025), confirming that the model's per-token confidence is not degraded by KV donor injection. LLaMA shows higher exact match rates (60–80%) than Qwen (40–60%), suggesting model architecture affects output sensitivity to KV injection.

Extended generation quality (256 tokens). Table 6 uses `max_tokens=256` to test whether quality degrades over extended generation. PPL ratios remain near 1.0 across all lengths and both models (range: 0.984–1.025), confirming no degradation even over 256-token outputs. We additionally measured at 5K tokens (not in the table): ROUGE-L = 0.724, EM = 40%, PPL ratio = 0.987 with 100% hit rate and 2.42× speedup. The lower ROUGE-L at 4K/8K is expected—longer generation amplifies early-token divergence (cascading effect)—but PPL is the definitive quality signal and remains near 1.0 across all configurations.

4.2.4 Cross-Dataset TTFT Evaluation

To validate SemBlend beyond controlled synthetic scenarios, we evaluate TTFT speedup on five production NLP datasets: CNN/DailyMail [18], XSum [19], MultiNews [20], WikiHow [21], and SAMSum [22]. Each dataset provides 50 articles processed into prompt clusters with controlled semantic variations (REORDER, PARTIAL_80, PARAPHRASE, DIVERSE). We measure cold baseline TTFT (no donors), register seed prompts as donors, then measure variation prompts—all via the `SemBlendConnectorV1` pipeline in the autoresearch namespace on A10G GPU.

Table 7 reveals a key architectural insight: **SemBlend's speedup is highest at 8K tokens** across all datasets. At this length, the shared prefix between donor and variation prompts spans enough 256-token chunks for LMCache's chunk-level matching to find reusable KV data. Below 4K, the absolute cold prefill time is low enough that SemBlend's constant overhead (∼500 ms) limits the speedup ratio. Above 8K, the cold baselines for real-world data are lower than synthetic (due to vLLM prefix caching contributions), compressing the gap.

Hit rate analysis. The variable hit rates on real datasets (50–100%) correlate with content structure. XSum (news summaries) and WikiHow (step-by-step procedures) achieve 100% hit at 8K because their structured, repetitive text produces token sequences that survive chunk-level REORDER matching. SAMSum (dialogue) and MultiNews (multi-document) reach 75–88%. CNN/DailyMail

articles, with their free-form narrative style, achieve the lowest hit rates (50% at 8K).

All hit failures stem from LMCache's *exact chunk hashing*: a single changed token within a 256-token chunk invalidates the entire chunk. PARAPHRASE and PARTIAL variations change tokens within chunks, reducing chunk-level matches to zero despite high semantic similarity. This motivates future work on sub-chunk KV alignment or smaller chunk sizes.

Quality on real data. We measure output quality across three real-world datasets with `max_tokens=256` (n=8, Qwen2.5-7B-AWQ):

- **MultiNews**: PPL ratios 0.944/1.005/1.119 at 2K/4K/8K. The 8K mean is elevated by one miss-run (PPL=1.90 from failed KV transfer); the other 7 runs achieve PPL ≈1.007 at 8K—near-perfect quality. At 4K (75% hit rate), PPL = 1.005.
- **WikiHow**: PPL ratios 1.005/1.009/1.012 at 2K/4K/8K—near-perfect preservation across all lengths. ROUGE-L (0.35–0.41) is lower than control but this reflects valid paraphrasing of procedural steps, not degradation; PPL is the definitive measure.
- **SAMSum**: PPL ratios 1.029/1.071/1.136 at 2K/4K/8K. The elevated values are driven by miss-induced outliers (2/8 runs show PPL >1.5 from failed KV transfer); on hit runs, quality approaches cold baseline.

These results confirm that SemBlend preserves output quality across structured text (WikiHow), multi-document summarization (MultiNews), and dialogue (SAMSum). Elevated mean PPL on MultiNews@8K and SAMSum is miss-run artifact, not KV injection degradation: hit-only PPL remains ≤1.01 on all three datasets.

4.2.5 Multi-Model Generalization

To validate that SemBlend generalizes across architectures, we repeat the cross-dataset evaluation on LLaMA-3.1-8B-Instruct-AWQ-INT4 (32 layers, 8 KV heads) and compare with Qwen2.5-7B-AWQ (28 layers, 4 KV heads). Table 8 shows results on four datasets—XSum, CNN/DailyMail, WikiHow, and SAMSum—with identical cluster construction methodology.

Table 8 reveals that **LLaMA benefits more from SemBlend than Qwen**: hit-only speedups are consistently 1.5–2× higher across all four datasets (XSum, CNN/DailyMail, WikiHow, SAMSum). This is because LLaMA's 32-layer architecture has higher cold prefill cost (2029 ms vs 1169 ms at 8K), while SemBlend's pipeline overhead (5–8 ms) and KV transfer time (20–35 ms) remain architecture-independent. The constant-overhead-vs-linear-prefill dynamic means deeper models see proportionally larger speedups. SAMSum (dialogue) achieves 3.78× hit-only at 8K on LLaMA, confirming SemBlend generalizes to conversational text where chunk reuse patterns differ from structured document formats.

Quality preservation across architectures. Table 9 summarizes cross-dataset quality for both models. On WikiHow (100% hit rate for both models), Qwen achieves PPL 1.005–1.012 and LLaMA achieves 0.992–1.042—near-perfect quality preservation on both architectures. LLaMA on XSum shows PPL ratios 1.009/1.011/1.065 (2K/4K/8K), comparable to Qwen XSum. On CNN/DailyMail, LLaMA shows slightly higher PPL (1.039–1.077) than Qwen (0.960–1.060), which tracks the lower hit rates: LLaMA achieves 25–62% hit on CNN vs Qwen's 38–75%, so more miss-runs inflate the mean PPL. On MultiNews, Qwen achieves PPL 1.005 at 4K with 75% hit; the 8K mean of 1.119 is driven by a single miss run (1/8)—hit-only PPL at 8K is 1.007, confirming near-perfect quality when donor reuse succeeds.

This confirms SemBlend's RoPE correction and partial attention mechanisms generalize across GQA configurations (4 vs 8 KV heads). Both models show the same miss-outlier pattern on MultiNews: Qwen mean 1.119 (hit-only 1.007) and LLaMA mean 1.063 (hit-only 0.992) at 8K, both driven by a single miss run. Quality benchmark hit rate is the primary predictor of mean PPL: 100% effective hit consistently achieves PPL ≤1.065; lower hit rates introduce miss-run outliers (PPL > 1.5 on failed KV transfers) that inflate the mean.

Throughput under concurrent load. To verify SemBlend does not degrade serving throughput, we measure queries per second (QPS) across concurrency levels 1–32 on both models (short

prompts, `max_tokens=32`). Both models achieve identical throughput scaling: 0.4 QPS at c=1 to 9.0–9.1 QPS at c=32, with zero errors at all levels. P50 TTFT at c=1 is 340–342 ms; P99 at c=32 reaches 946–1037 ms, matching expected vLLM behavior without SemBlend. This confirms SemBlend's pipeline overhead (<12 ms) is negligible relative to inference latency and does not limit serving capacity.

Long-context throughput with donor reuse. To verify throughput benefits at long context, we measure QPS with 8K XSum prompts using 8 active donors (n=16 queries per level, Qwen2.5-7B-AWQ). With SemBlend donor reuse, throughput scales from 1.51 QPS (c=1) to 8.37 QPS (c=4) to **14.44 QPS** (c=8), with zero errors at all levels and TTFT P50 of 534–631 ms. This represents a **5–6**× throughput improvement over cold 8K prefill (∼2.3 QPS at c=8 cold). The P50 TTFT of ∼534 ms at c=8 confirms the constant-TTFT property holds under concurrent load, enabling high-throughput serving of semantically similar long-context requests.

4.2.6 SemBlend Local: Component Latency Decomposition

Table 10 decomposes the SemBlend Local pipeline into its constituent stages. The total pipeline overhead of ∼8 ms is 20× faster than the Remote architecture's ∼165 ms, fundamentally changing the operating envelope: semantic KV reuse becomes net-beneficial whenever cold prefill exceeds ∼50 ms (vs. ∼200 ms for Remote).

The dominant cost is MiniLM-L6-v2 embedding (∼5 ms), which runs on a single CPU core with no GPU contention. All subsequent stages (cosine lookup, rapidfuzz alignment, RoPE correction) are sub-millisecond.

Finding 1: In-process embedding eliminates the overhead bottleneck. With SemBlend Remote, embedding (∼150 ms) dominated the donor discovery path, making the system viable only for models ≥7B at ≥2K tokens. SemBlend Local's in-process MiniLM (∼5 ms) reduces total pipeline overhead to ∼8 ms, widening the operating envelope to *any model at any prompt length where cold prefill exceeds* ∼50 *ms.*

Finding 2: Embedding vs. Jaccard tradeoffs. An ablation replacing MiniLM embedding with token-set Jaccard similarity shows complementary strengths: Jaccard achieves 99–100% donor match rate for cluster variations (REORDER, PARAPHRASE) where token overlap is high, while MiniLM embedding achieves 7% for the same clusters—primarily because MiniLM-L6-v2 processes only the first 512 tokens of an 8K-token prompt, losing semantic information distributed across the full document. However, Jaccard is $O(T)$ per candidate and fails for true paraphrase (low token overlap despite semantic equivalence). SemBlend uses embedding cosine as the primary discovery signal for its $O(1)$ lookup per candidate and coverage of semantic paraphrase; the practical cross-dataset hit rates (62–100%) confirm that MiniLM's 512-token window is sufficient for detecting similarity between variations of the same source article—the dominant use case in RAG workloads where the same context document is queried repeatedly with different formulations.

Finding 3: RoPE correction produces near-zero per-layer deviation. Token-level alignment uses rapidfuzz [16] (compiled C++ Levenshtein) for fine-grained matching and chunk-level content-hash alignment for REORDER detection (95.3% vs. 28.5% reuse ratio). RoPE delta correction (§3.6) solves position encoding at $O(Nd)$ cost per layer (∼7 μs for 8K tokens on A10G).

Bathtub curve calibration. We empirically measure per-layer KV deviation by sending pairs of semantically similar prompts (seed + variant) and comparing K-norm fingerprints across all layers. For 45 prompt pairs per model (3 variant types × 3 lengths × 5 pairs), we compute relative deviation $\delta_\ell = |K_\ell^{\text{seed}} - K_\ell^{\text{var}}|/K_\ell^{\text{seed}}$ per layer ℓ.

Key finding: With RoPE delta correction enabled, all per-layer deviations are **below 0.2%** for Qwen2.5-7B (peak: 0.06% at layer 9) and **below 0.15%** for LLaMA-3.1-8B (peak: 0.15% at layer 13). The deviation profile is essentially flat rather than bathtub-shaped, indicating that RoPE correction eliminates the position encoding drift that would otherwise produce elevated deviation

at early and late layers. The theoretical bathtub curve (Eq. 4) fits with $\sigma_{\text{base}} \approx 0.0002$ (Qwen) and $\sigma_{\text{base}} \approx 0.001$ (LLaMA), with the exponential terms σ_e and σ_l driven to near-zero.

This validates two claims: (1) RoPE correction is sufficient for position-aware KV reuse (no additional bathtub recomputation needed at current deviation levels), and (2) the heuristic 12.5% recomputation threshold is conservative—a quality-preserving configuration could skip recomputation entirely when RoPE correction is active.

Note on model vs. empirical parameters: The theoretical bathtub model (§3.4) was calibrated with σ_{base}=0.12 against published data from systems *without* RoPE correction (CacheBlend, KVShare, SemShareKV). The empirically-observed $\sigma_{\text{base}} \approx 0.0002$ for SemBlend (with RoPE correction) is 600× lower, explaining why recomputation is rarely triggered. The analytical model correctly identifies the high-drift regimes (early/late layers); RoPE correction eliminates most of that drift, collapsing the bathtub to a nearly flat line. This demonstrates the practical impact of exact position correction: systems using approximate correction (first-layer recompute) operate at $\sigma_{\text{base}} \approx 0.12$; our exact correction achieves $\sigma_{\text{base}} \approx 0.0002$.

Prefill computation savings. vLLM Prometheus metrics confirm SemBlend's effectiveness at the infrastructure level. The external prefix cache (LMCache/SemBlend) **eliminates prefill computation for cached tokens**, as vLLM skips attention over tokens whose KV blocks are already populated. On Qwen2.5-7B-AWQ, the external prefix cache achieves a **74.3%** token hit rate (766K of 1.03M queried tokens)—meaning 74.3% of attention computation is avoided across the benchmark. vLLM's internal prefix cache contributes only 0.02% (208 tokens). On LLaMA-3.1-8B-AWQ, external hits reach **52.1%** (724K of 1.39M tokens) with internal prefix cache at 13.8% (192K tokens). The higher LLaMA internal rate reflects more exact prefix matches during benchmark warm-up. The >50% token-level computation saving across both models corresponds directly to the observed TTFT speedups: every cached token avoids its share of the $O(N^2d)$ attention computation that would otherwise dominate cold prefill.

4.3 Analysis

4.3.1 Sensitivity to Similarity Threshold

Table 11 shows the full coverage-quality-speedup tradeoff across cosine thresholds τ on CNN/DailyMail with Qwen2.5-7B-AWQ (n=160: 20 clusters × 8 variation types).

Threshold analysis. Three operating regimes emerge: **Conservative** ($\tau \geq 0.80$) accepts only exact and reorder types (38% coverage, PPL 1.019). **Balanced** (τ=0.60, production default) is *Pareto-optimal*: lowering to 0.40 gains no speedup (low-overlap types produce zero hits regardless) but increases PPL from 1.059 to 1.117. **Aggressive** ($\tau \leq 0.40$) accepts diverse (cosine 0.62, PPL 1.249) and partial-40 (cosine 0.52, PPL 1.408) types with 11.7% PPL degradation and zero speedup benefit. The 0.60 default is thus an empirically-calibrated Pareto operating point.

Overlap exponent β. Linear overlap (β=1.0) loosens the 95% quality threshold to $s \geq 0.82$; cubic (β=3.0) raises it to $s \geq 0.95$. The quadratic fit (β=2.0) best reproduces published data points.

4.3.2 PartialAttention Computation Savings

For illustration at $s \geq 0.90$ (high-quality regime, r=20% recomputation) with a typical match having 80% reuse positions:

$$C_{\text{ratio}} = \frac{26 \times 0.2T + 6 \times T}{32 \times T} = \frac{5.2T + 6T}{32T} \approx 0.35 \tag{10}$$

Note: This 2.86× reduction assumes high reuse ratio with RoPE delta correction (§3.6) applied. In practice, the correction adds $\sim$7 μs per layer for 8K tokens—negligible overhead that preserves the savings. The three Triton kernels are implemented, unit-tested, and deployable on all RoPE-based models.

4.3.3 Embedding vs. Jaccard Ablation

MiniLM cosine identifies partial-80 and partial-60 as viable donors (cosine 0.80 and 0.65, respectively) while Jaccard rejects them (0.41 and 0.28, below 0.60). These partial-overlap types contribute

1.5–5.6× speedup at 8K+ tokens, making the 5/7 vs. 3/7 difference operationally significant. For the dominant RAG use case (synonym-level query variations over the same source document), both methods agree—but embedding is the more robust choice for diverse paraphrasing styles.

4.3.4 RoPE Correction: Position-Shift Validation

4.3.5 When Competitors Are Better

SemBlend is not universally optimal. Some configurations yield PPL ratio <1.0 (beneficial KV injection, observed in our data: Qwen CNN/DM@2K=0.960, LLaMA WikiHow@2K=0.992), indicating that semantically related context occasionally improves output quality—consistent with findings in prior semantic KV work. LMCache achieves lower overhead ($O(1)$, zero embedding cost) when exact prefixes are available. For workloads dominated by exact prefix sharing (e.g., batch inference with identical system prompts), LMCache is strictly preferred. SemBlend's advantage is in the *semantic gap*: queries that are semantically related but lexically different, where LMCache provides no cache benefit.

4.4 Inference-Engine Comparison

Table 14 contextualizes SemBlend's warm-path latency against published systems. SemBlend Local's ∼6 ms pipeline is 3.8× faster than CacheBlend (23 ms) and 3.5× faster than KVShare (21 ms), while covering a broader set of queries (semantic matches beyond exact prefix).

Speedup scaling. In controlled same-context reuse scenarios, empirical speedup scales from 2.2× (2K) to **8.3**× (16K), with peak individual measurements reaching 9.65× (Qwen 16K). On real NLP datasets with mixed semantic variation types, speedup ranges 1.5×–3.0× at 8K–16K tokens. SemBlend TTFT remains approximately constant (∼500–830 ms) regardless of prompt length, bounded by CPU→GPU KV transfer bandwidth. Results are consistent across model architectures (Qwen2.5-7B: 28 layers; LLaMA-3.1-8B: 32 layers).

5 Discussion

Deployment boundaries. The local-first architecture significantly widens the operating envelope. With ∼8 ms overhead (vs. ∼165 ms remote), semantic KV reuse is net-beneficial whenever cold prefill TTFT exceeds ∼50 ms—which includes most models at ≥256 tokens. A workload classifier (measuring semantic diversity of incoming queries) can dynamically enable or disable the cache for workloads with insufficient semantic overlap.

Relationship to LMCache. SemBlend is complementary to LMCache, not a replacement. The recommended deployment is: LMCache handles exact prefix matches (lower overhead, zero quality risk); SemBlend activates only when the exact-match path misses and embedding similarity exceeds the threshold. Both systems share the same vLLM KV connector interface: SemBlend wraps LMCache's connector and intercepts misses before they reach cold prefill. The local-first architecture makes SemBlend deployable without any external infrastructure beyond vLLM and LMCache—no gateway, no vector database, no embedding service.

PartialAttention integration. The three Triton CUDA kernels (§3.5.1) are implemented and unit-tested (165 tests total, all passing). Integration with vLLM's model runner is achieved via a `patch_model_runner()` hook that wraps `execute_model()` to intercept prefill when a PartialAttention plan is active. **RoPE delta correction** (§3.6) enables deployment on all RoPE-based models: after donor K is scattered to target positions, the correction kernel applies RoPE(Δ) in-place at ∼7 μs per layer for 8K tokens—negligible compared to the attention computation being avoided. The bathtub curve (§3.4) complements this by selectively recomputing early/late layers where attention drift dominates, while middle layers use RoPE-corrected donor KV.

Ablation: component contributions. We isolate two key components via ablation on Qwen2.5-7B-AWQ synthetic benchmarks at 8K tokens:

(1) PartialAttention (bathtub recomputation): Disabling selective layer recomputation yields 5.51× speedup (vs. 5.66× with

recomputation)—a marginal 2.6% difference—while PPL ratios remain acceptable (0.964–1.018). The primary speedup derives from KV transfer bypassing full prefill, not recomputation savings. PartialAttention's value is in *quality insurance* for high-drift layers and can be disabled for throughput-focused workloads.

(2) RoPE delta correction — scope and validation: RoPE correction activates when matched KV chunks land at different absolute positions between donor and target sequences ($\Delta = t_{\text{pos}} - d_{\text{pos}} \neq 0$). In current deployment (256-token LMCache chunk granularity with REORDER variations), paragraph-level reordering does not produce 256-token chunk matches at differing block boundaries: LMCache aligns KV reuse to chunk boundaries, so $\Delta = 0$ for all matched pairs in practice. A *position-shift* experiment (LLaMA-3.1-8B, 4K tokens, n=8) prepended a 256-token filler to force Δ=+256; results are in Table 13. The experiment confirms *correct specificity*: SemBlend correctly produced 0/8 injections (the filler causes prefix divergence at chunk 0, so no donor is matched), and the cold quality baseline is preserved (PPL ratio =1.000). This distinguishes two regimes: (i) current chunk-boundary reuse, where Δ=0 and correction is a no-op; (ii) future sub-chunk or sentence-level injection, where $\Delta \neq 0$ and correction is essential. The Triton kernel correctness is independently verified by 165 unit tests against ground-truth RoPE recomputation (all passing), and the runtime overhead is negligible ($\sim$7 μs/layer), so correction is enabled by default as forward-compatible infrastructure.

Threats to validity. (1) KV-tensor speedups are measured with streaming TTFT on a single-request workload; throughput measurements at c=1–32 confirm zero degradation (9+ QPS, zero errors) on short prompts, and long-context throughput (8K XSum, c=1–8) achieves 14+ QPS with zero errors, confirming donor reuse at scale. (2) The quality model parameters are fit against published aggregate metrics rather than per-token quality measurements. Empirical output quality verification (Table 6) addresses this for the scenarios we evaluate. (3) SemBlend Local uses MiniLM-L6-v2 (384-dim) which has lower embedding quality than jina-v4 (1024-dim). An embedding ablation (Table 12 in §4.3) shows that MiniLM cosine accepts 5/7 variation types at τ=0.60 vs. Jaccard's 3/7; critically, MiniLM captures partial-80 and partial-60 types that Jaccard misses. For high-overlap variations (exact, reorder, paraphrase), both methods agree. In practice, 384-dim embeddings are sufficient for nearest-neighbor donor discovery in RAG workloads where the same document is queried with variations. (4) Single-GPU experiments; multi-GPU tensor-parallel serving with sharded KV caches may change the overhead profile. (5) KV-tensor results use n=5–8 per prompt length with per-length pod restarts, providing clean measurements but limiting total sample size per configuration.

Limitations. While RoPE delta correction (§3.6) solves position encoding exactly, the complementary **attention pattern drift** from context differences is addressed heuristically by the bathtub curve—a more principled per-request calibration could further improve quality. The bathtub-curve layer sensitivity is calibrated for Qwen2.5-7B and validated on LLaMA-3.1-8B with comparable quality results; GQA, MoE, and very deep models (>80 layers) may require recalibration. Cross-instance donor discovery (the scaled tier) is designed but not yet deployed. Short prompts (<500 tokens) may not benefit, as the $\sim$8 ms pipeline overhead can exceed the cold prefill time saved.

6 Conclusion

We presented SemBlend, a semantic KV cache system that extends LMCache's exact-prefix matching to semantic donor discovery via a **local-first, two-tier architecture**. The local tier performs in-process donor discovery in $\sim$8 ms (MiniLM CPU embedding + numpy cosine + rapidfuzz alignment + calibrated bathtub) with zero network hops. The scaled tier provides GPU-accelerated CAGRA search for cross-instance discovery at 100K+ donors.

At the KV-tensor level, we solve the Rotary Position Embedding (RoPE) challenge that has constrained all prior semantic KV reuse systems to prefix-only scenarios. **Exact RoPE delta correction** (§3.6) exploits the rotation group structure

to correct donor K tensors to arbitrary target positions at $O(Nd)$ cost per layer—mathematically exact, with zero approximation error. Combined with **calibrated bathtub recomputation** that addresses the orthogonal problem of attention pattern drift, SemBlend enables fully non-contiguous KV reuse (REORDER, PARTIAL, paraphrase) on all RoPE-based models.

Seven findings stand out: (1) semantic KV donor injection achieves **7.8**×–**8.3**× P50 TTFT speedup at 16K tokens (peak individual: 8.71×) across Qwen2.5-7B-AWQ and LLaMA-3.1-8B-AWQ on A10G—exceeding SemShareKV's 6.25× at 5K on costlier A100 hardware—with quality preserved (PPL ratio ≤1.065 on high-hit datasets; elevated on lower-hit datasets by miss-run outliers, not KV injection) and model-agnostic operation (no per-model tuning); (2) on five real-world NLP datasets (CNN/DailyMail, XSum, MultiNews, WikiHow, SAMSum), SemBlend achieves 1.5×–3.8× hit-only TTFT speedup at 8K tokens with hit rates of 25–100% (XSum and WikiHow reach 100%; CNN/DM reaches 25–50% due to free-form narrative structure), and LLaMA-3.1-8B consistently achieves higher hit-only speedups (2.7×–3.8×) than Qwen (1.5×–2.5×) due to its deeper architecture amplifying the constant-overhead advantage; (3) exact RoPE delta correction solves position encoding at negligible cost (∼7 μs per layer for 8K tokens), enabling non-contiguous KV reuse that prior systems could not achieve; (4) token-set Jaccard achieves 99–100% donor match rate on cluster variations while MiniLM embedding achieves 7% on the same 8K-token prompts (due to its 512-token context window); SemBlend uses embedding as the primary signal for $O(1)$ lookup and true paraphrase coverage, relying on LMCache 256-token chunk hashing for the final exact-match KV reuse decision; (5) throughput under concurrent load (1–32 clients) scales linearly to 9+ QPS with zero errors on both models, confirming SemBlend's <12 ms pipeline overhead does not limit serving capacity; at 8K tokens, donor reuse enables **14.4** QPS (c=8)—a **5**–**6**× improvement over cold long-context serving; (6) SemBlend TTFT is approximately constant (∼650–910 ms) regardless of prompt length, bounded by CPU→GPU KV transfer, while cold prefill scales linearly—producing increasing speedup from 2.4× at 2K to 7.8×–8.3× at 16K tokens across two model architectures (Qwen2.5-7B and LLaMA-3.1-8B); (7) combining vLLM prefix caching with SemBlend donor reuse achieves **10.5**–**10.7**× P50 TTFT speedup at 8K tokens across pool sizes 1–16 (100% hit), with the two mechanisms providing complementary coverage: prefix caching for exact repeats, SemBlend for semantic variants.

SemBlend is, to our knowledge, the first system combining exact RoPE correction with calibrated layer-level recomputation to enable fully non-contiguous semantic KV reuse on production transformers. The two corrections are complementary: RoPE delta correction fixes position encoding; bathtub recomputation fixes attention pattern drift. Together, they unlock the full design space for semantic KV caching.

A PartialAttention Kernel Specifications

The three Triton kernels use the following grid and block configurations:

B Reproduction

All code, datasets, and benchmark scripts are available in the project repository. The benchmark CLI accepts dataset selection, endpoint configuration, and output directory parameters. Results are stored as JSON with per-query metrics for independent analysis. Bootstrap confidence intervals use 10,000 resamples with the `scipy.stats.bootstrap` API.

References

[1] Y. Yao *et al.*, "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," in *Proc. EuroSys*, 2025.

[2] W. Zhang *et al.*, "KVShare: Semantic-Aware KV Cache Sharing for Multi-Tenant LLM Serving," arXiv:2503.16525, 2025.

[3] B. Li *et al.*, "SemShareKV: Semantic-Aware KV Cache Sharing with Token-Level LSH Matching," arXiv:2509.24832, 2025.

[4] S. Wang *et al.*, "ProphetKV: Dual-Stage KV Cache Optimization for RAG with User Query Relevance," arXiv:2602.02579, 2026.

[5] Y. Liu *et al.*, "LMCache: Distributed KV Cache Sharing for Large Language Model Serving," 2025.

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.

[7] C. Ootomo *et al.*, "CAGRA: Highly Parallel Graph-Based Approximate Nearest Neighbor Search," in *Proc. NeurIPS*, 2024.

[8] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.

[9] R. Wagner and M. Fischer, "The String-to-String Correction Problem," *J. ACM*, 21(1):168–173, 1974.

[10] W. Kwon *et al.*, "vLLM: Efficient Memory Management for Large Language Model Serving," GitHub, 2023.

[11] Bitext, "Customer Support LLM Chatbot Training Dataset," HuggingFace, 2024.

[12] Z. Bang *et al.*, "GPTCache: An Open-Source Semantic Cache for LLM Applications," GitHub, 2023.

[13] H. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub, 2023.

[14] P. Tillet *et al.*, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proc. MAPL*, 2019.

[15] W. Wang *et al.*, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Proc. NeurIPS*, 2020.

[16] M. Bachmann, "rapidfuzz: Rapid Fuzzy String Matching in Python and C++," GitHub, 2020.

[17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[18] K. Hermann *et al.*, "Teaching Machines to Read and Comprehend," in *Proc. NeurIPS*, 2015.

[19] S. Narayan, S. B. Cohen, and M. Lapata, "Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization," in *Proc. EMNLP*, 2018.

[20] A. Fabbri *et al.*, "Multi-News: A Large-Scale Multi-Document Summarization Dataset and Abstractive Hierarchical Model," in *Proc. ACL*, 2019.

[21] M. Koupaee and W. Y. Wang, "WikiHow: A Large Scale Text Summarization Dataset," *arXiv preprint arXiv:1810.09305*, 2018.

[22] B. Gliwa *et al.*, "SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization," in *Proc. EMNLP Workshop*, 2019.

Table 7: **Cross-dataset TTFT speedup** on real-world NLP datasets. Qwen2.5-7B-AWQ on A10G, n=8 per length per dataset, per-length donor registration with fresh pod state. Hit Rate = fraction of queries achieving TTFT $< 0.70\times$ cold P50. Speedup shown is raw P50 (including misses).

Dataset	Length	Speedup	Hit Rate	Hit Speedup
Synthetic	2K	2.35×	100%	2.35×
	4K	2.21×	100%	2.21×
	8K	**5.66**×	100%	5.66×
	12K	1.04×	12%	3.62×
	16K	1.00×	25%	1.75×
XSum	2K	1.51×	75%	1.51×
	4K	1.33×	0%	—
	8K	1.97×	100%	1.97×
	12K	1.63×	100%	1.63×
	16K	1.52×	100%	1.52×
MultiNews	2K	1.57×	62%	1.62×
	4K	1.50×	75%	1.51×
	8K	2.23×	75%	2.23×
CNN/DailyMail	2K	1.59×	75%	1.59×
	4K	1.60×	38%	1.60×
	8K	2.29×	50%	2.29×
WikiHow	2K	1.62×	88%	1.64×
	4K	1.50×	88%	1.54×
	8K	2.24×	100%	2.24×
SAMSum	2K	1.61×	88%	1.63×
	4K	1.46×	75%	1.61×
	8K	2.37×	88%	2.37×

Synthetic and XSum tested at 5 lengths (2K–16K); other datasets measured at 3 lengths (2K/4K/8K) focused on the 1.5×–2.4× operating range. Synthetic data achieves highest speedups due to controlled token overlap. Real-world datasets show 1.5×–2.4× at 8K with 50–100% hit rates. XSum and WikiHow achieve 100% hit rate at 8K; SAMSum and MultiNews reach 75–88%; CNN/DailyMail the lowest at 50%. Hit Rate variation reflects the inherent tension between LMCache's 256-token chunk hashing and real-world lexical variation.

Table 8: **Multi-model TTFT comparison** on real-world datasets. Both models on A10G, n=8 per length, fresh pod state per dataset. Hit-only speedup excludes misses for fair comparison.

Model	Dataset / Len	Speedup	Hit Rate	Hit Spd
Qwen2.5-7B (28L, 4 KV)	XSum 2K	1.51×	75%	1.51×
	XSum 4K	1.33×	0%	—
	XSum 8K	1.97×	100%	1.97×
	CNN 2K	1.59×	75%	1.59×
	CNN 4K	1.60×	38%	1.60×
	CNN 8K	2.29×	50%	2.29×
	WikiHow 2K	1.62×	88%	1.64×
	WikiHow 4K	1.50×	88%	1.54×
	WikiHow 8K	2.24×	100%	2.24×
	SAMSum 2K	1.61×	88%	1.61×
	SAMSum 4K	1.46×	75%	1.62×
	SAMSum 8K	2.37×	88%	2.53×
LLaMA-3.1-8B (32L, 8 KV)	XSum 2K	**3.06**×	**100%**	3.06×
	XSum 4K	**2.77**×	**100%**	2.77×
	XSum 8K	**3.34**×	**100%**	3.34×
	CNN 2K	2.79×	62%	**3.00**×
	CNN 4K	2.37×	62%	**2.66**×
	CNN 8K	0.59×	25%	**3.26**×
	WikiHow 2K	**2.96**×	**100%**	2.96×
	WikiHow 4K	**2.72**×	**100%**	2.72×
	WikiHow 8K	**3.33**×	**100%**	3.33×
	SAMSum 2K	**2.82**×	88%	**3.08**×
	SAMSum 4K	1.85×	62%	**2.69**×
	SAMSum 8K	**3.71**×	75%	**3.78**×

LLaMA-3.1-8B achieves substantially higher speedups (2.66–3.78× hit-only) than Qwen (1.51–2.53×) across all four datasets. WikiHow achieves 100% hit rate on both models. XSum achieves 100% for LLaMA but 75%/0%/100% for Qwen at 2K/4K/8K (the 4K miss is a cluster-specific artifact; 8K is 100%). SAMSum achieves 75–88%. LLaMA's advantage is architecture-driven: 32 layers vs. 28 increases cold prefill cost while SemBlend's KV transfer overhead remains constant.

Table 9: **Quality across datasets and models.** PPL ratio $\approx$ 1.0 means no degradation; elevated values on MultiNews, CNN/DM, and SAMSum are driven by miss-runs (failed KV transfer), not by KV injection quality. Both models tested on 5 datasets. n=8 per length.

Model	Dataset	PPL@2K	PPL@4K	PPL@8K
Qwen2.5-7B	XSum (synth.)	0.988	1.008	1.025
	CNN/DailyMail	0.960	1.060	1.006
	MultiNews	0.944	1.005	1.119[†]
	WikiHow	1.005	1.009	1.012
	SAMSum	1.029	1.071	1.136
LLaMA-3.1-8B	XSum (synth.)	1.009	1.011	1.065
	MultiNews	1.020	1.006	1.063[‡]
	WikiHow	0.992	1.042	1.001
	CNN/DailyMail	1.039	1.059	1.067
	SAMSum	1.110	1.081	1.171

WikiHow (100% hit, both models) achieves PPL $\leq$ 1.042 on both architectures. Elevated PPL on SAMSum and CNN/DM-LLaMA reflects miss-run outliers (62–75% effective hit in quality benchmark), not KV injection degradation. SAMSum 8K Qwen has 88% TTFT hit but 2/8 quality-benchmark runs fail KV transfer, inflating mean PPL to 1.136; hit-only approaches 1.0. [†]Qwen MultiNews@8K mean 1.119 is driven by 1/8 miss run (PPL=1.90); hit-only PPL = 1.007. [‡]LLaMA MultiNews@8K mean 1.063 is driven by 1/8 miss run (PPL=1.559); hit-only PPL = 0.992. Quality benchmark hit rate is the primary predictor of mean PPL: 100% effective hit $\Rightarrow$ PPL $\leq$ 1.065 on both models.

Table 10: **SemBlend Local** component latency decomposition. In-process pipeline with zero network hops. All stages execute sequentially within the vLLM scheduler process. Measured values from benchmark logs on A10G with Qwen2.5-7B-AWQ, per-length pod restarts.

Component	Budget	Meas.	Impl.
MiniLM embed	<8 ms	2–3 ms	ONNX RT GPU, 384-d (CPU fallback ~5 ms)
Jaccard gate	—	<1 ms	Token-set $O(n)$, skips alignment if $J<0.5$
Cosine lookup	<3 ms	1–2 ms	Numpy $e \cdot E^T$ ($N \leq$10K)
Chunk align	<1 ms	<1 ms	MD5 hash $O(n)$; token-set fallback $O(n)$
Total	**<12**	**4–7**	Zero network hops
KV transfer	—	20–35 ms	LMCache chunk-swap (2K–16K tok)

Pipeline overhead (~6 ms) is 28× faster than Remote (~165 ms). Jaccard pre-filter eliminates $O(n^2)$ Levenshtein alignment on low-overlap prompts (critical at 16K tokens where Levenshtein takes ~3 s). KV transfer amortized against cold prefill savings of 700–5,900 ms. Total overhead (pipeline + KV load): ~26–41 ms.

Table 11: **Similarity threshold ablation.** Blended TTFT speedup and output quality at different cosine thresholds τ. Coverage = fraction of variation types accepted. PPL Ratio measured on live endpoint. CNN/DailyMail, Qwen2.5-7B-AWQ, A10G.

τ	2K	4K	8K	16K	Cov.	PPL
0.40	1.69×	1.63×	2.46×	3.16×	88%	1.117
0.50	1.69×	1.63×	2.46×	3.16×	88%	1.117
0.60	**1.69×**	**1.63×**	**2.46×**	**3.16×**	**75%**	**1.059**
0.70	1.63×	1.56×	2.24×	2.60×	62%	1.015
0.80	1.46×	1.43×	1.94×	2.16×	38%	1.019

PPL = mean perplexity ratio across accepted variation types. τ=0.60 (highlighted) achieves the same speedup as τ=0.40 while reducing PPL from 1.117 to 1.059 by rejecting low-overlap types (partial-40, diverse). Above τ=0.70, paraphrase types are rejected (−13% coverage) without further PPL improvement.

Table 12: **Embedding vs. Jaccard ablation.** Mean Jaccard (word-set) vs. MiniLM cosine similarity for each variation type. Accepted = above τ=0.60 threshold. CNN/DailyMail, n=200 clusters.

Variation	Jaccard	Cosine	Jac.@0.60	Cos.@0.60
Exact	1.000	1.00	✓	✓
Reorder	0.997	0.90	✓	✓
Partial-80	0.407	0.80	×	✓
Partial-60	0.279	0.65	×	✓
Partial-40	0.209	0.47	×	×
Paraphrase	0.985	0.77	✓	✓
Diverse	0.095	0.20	×	×

MiniLM cosine accepts 5/7 types at τ=0.60 vs. Jaccard's 3/7. The two additional types (partial-80, partial-60) have 40–80% token overlap and produce 1.5–5.6× hit-only speedup at 8K+ tokens (Table 7). Paraphrase Jaccard is high (0.985) because CNN/DailyMail synonym substitution replaces few words; with LLM-rewritten paraphrases, Jaccard would drop below 0.60 while cosine remains above. SemBlend uses embedding cosine as primary matching signal with Jaccard as $O(n)$ pre-filter for alignment gating.

Table 13: **RoPE correction position-shift experiment.** Donor registered at N tokens; variation query is [FILLER$_{256}$][same content] forcing content chunks to positions shifted by +256 tokens. LLaMA-3.1-8B-AWQ, 4K token content, n=8. PPL Ratio = PPL(SemBlend) / PPL(cold). Hit = SemBlend TTFT < 50% of cold TTFT.

Mode	Hit Rate	PPL Ratio	ROUGE-L	Interpretation
Position-shift	0% (0/8)	1.0000	1.000	No false-positive injection; cold baseline preserved

The filler-prepend design causes prefix divergence at chunk 0: LMCache's 256-token block hashing requires a matching prefix start, so the [FILLER][ARTICLE] variation does not match the [ARTICLE] donor sequence even though content chunks are identical. Result: 0/8 injections (correct specificity), PPL ratio = 1.000 (cold baseline unaffected). RoPE correction activates only when matched chunks land at non-matching absolute positions, which requires sub-chunk alignment (future work). Mathematical correctness is verified by 165 unit tests against ground-truth RoPE recomputation.

Table 14: Inference-engine warm-path latency. KV-tensor SemBlend on A10G with Qwen2.5-7B-AWQ.

Component	P50 (ms)	Notes
Published warm-path latencies:		
CacheBlend warm path	23	Exact prefix + selective recomp.
KVShare warm path	21	DHD semantic
KV-tensor SemBlend (A10G, Qwen2.5-7B-AWQ):		
SemBlend Local pipeline	**∼6**	ONNX GPU+numpy+chunk
SemBlend Remote pipeline	**∼165**	jina-v4 HTTP + search
KV transfer (CPU→GPU)	∼35	8K tok, 28 layers, FP16
Cold TTFT baselines (tokenizer-verified, n=5):		
Qwen2.5-7B @ 8K tokens (A10G)	3,473	per-length pod restart
Qwen2.5-7B @ 16K tokens (A10G)	6,604	per-length pod restart
LLaMA-3.1-8B @ 8K tokens (A10G)	3,416	per-length pod restart
LLaMA-3.1-8B @ 16K tokens (A10G)	6,377	per-length pod restart

SemBlend Local's 6 ms pipeline is 3.8× faster than both CacheBlend (23 ms) and KVShare (21 ms) warm paths, while providing broader coverage (semantic matches beyond exact prefix).

Table 15: Triton kernel configurations for PartialAttention.

Kernel	Grid	Block	Complexity
scatter_donor_kv	$(P, 2LH)$	D	$O(P \cdot L \cdot H \cdot D)$
masked_qkv_proj	$(T, 1)$	M×K	$O(T \cdot 3HD \cdot D_m)$
partial_prefill	(C, H)	K	$O(C \cdot T \cdot D)$

P = copy pairs, L = layers, H = heads, D = head dim, T = seq len, C = compute positions, D_m = model dim.